



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ
НА ТЕМУ:

«Разработка базы данных приложения по организации
археологических экспедиций»

Студент группы **ИУ7-63Б**

(Подпись, дата)

Завойских Е. В.

(И.О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Исаев А. Л.

(И.О. Фамилия)

2024 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой

(индекс)

Рудаков И.В.

(И.О. Фамилия)

3 марта 2024 г.

(дата)

(подпись)

З А Д А Н И Е
на выполнение курсовой работы

по дисциплине Базы Данных

Студент группы ИУ7-63Б Завойских Евгения Васильевна

(Фамилия, имя, отчество)

Тема курсовой работы Разработка базы данных приложения по организации
археологических экспедиций

Направленность КР (учебная, исследовательская, практическая, производственная, др.)
Учебная

Источник тематики (кафедра, предприятие, НИР) Кафедра

Задание

Провести анализ предметной области и сформулировать требования и ограничения к разрабатываемой базе данных и приложению. Сформулировать описание пользователей проектируемого приложения. Спроектировать сущности базы данных и ограничения целостности. Спроектировать ролевую модель на уровне базы данных. Выбрать средства реализации базы данных и приложения. Описать методы тестирования разработанного функционала и разработать тесты для проверки корректности работы приложения. Исследовать зависимость времени выполнения запроса от наличия индексов.

Оформление курсовой работы:

Расчетно-пояснительная записка (Отчет по КР) на 25-40 листах формата А4. Презентация на 12-18 слайдах.

Дата выдачи задания «5» марта 2024 г.

Руководитель курсовой работы

(подпись,
дата)

Исаев А.Л.

(И.О. Фамилия)

Студент

(подпись,
дата)

Завойских Е.В.

(И.О. Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

РЕФЕРАТ

Расчетно-пояснительная записка 39 с., 9 рис., 3 табл., 15 ист.

Цель работы: разработка базы данных приложения по организации археологических экспедиций.

Ключевые слова: базы данных, Golang, PostgreSQL, археологические экспедиции, раскопки, API.

В ходе выполнения работы был проведен анализ предметной области и сформулированы требования и ограничения к базе данных. Были спроектированы сущности базы данных и ролевая модель. Разработано приложение по организации археологических раскопок. Проведено исследование зависимости времени выполнения запроса от наличия индексов.

Содержание

ВВЕДЕНИЕ	6
1 Аналитический раздел	8
1.1 Формализация задачи	8
1.2 Формализация данных	8
1.3 Формализация ролей	10
1.4 Базы данных и системы управления базами данных	12
1.4.1 Дореляционные	12
1.4.2 Реляционные	13
1.4.3 Постреляционные	13
1.4.4 Выбор модели данных	13
2 Конструкторский раздел	15
2.1 Проектирование базы данных	15
2.2 Триггер базы данных	17
2.3 Роли базы данных	18
3 Технологический раздел	20
3.1 Выбор СУБД	20
3.2 Средства реализации	20
3.3 Детали реализации	21
3.3.1 Таблицы базы данных	21
3.3.2 Роли базы данных	24
3.3.3 Триггер базы данных	26
3.4 Тестирование	27
3.5 Интерфейс приложения	27
3.6 Примеры работы ПО	31

4	Исследовательский раздел	33
4.1	Технические характеристики	33
4.2	Цель и описание исследования	33
4.3	Результаты исследования	34
	ЗАКЛЮЧЕНИЕ	36
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37
	ПРИЛОЖЕНИЕ А Презентация	39

ВВЕДЕНИЕ

Назначение археологии можно было бы объяснить простым любопытством, которое толкает человека к познанию и открытиям. Не последнюю роль играет стремление человека к изучению своих истоков, своей культуры. Человек без прошлого — это человек без будущего. Совершенно определенно можно утверждать, что изучая прошлое, мы прогнозируем свое будущее.

Археология невозможна без проведения полевых исследований, в основной массе это археологические разведки и раскопки, где все работы проводятся вручную с использованием огромного арсенала спецсредств. Опыт раскопок обязателен для любого археолога, даже применяющего иные методы исследований, которые, в свою очередь, обретают свою значимость только благодаря раскопкам. Только раскопками можно проверить идеи, модели и теории.

Важность раскопок на современном этапе развития науки и общества можно проиллюстрировать минимум тремя примерами. Первый пример касается времени появления современных людей и их сосуществования с неандертальцами. Второй пример относится к объектам XX века — исследованию так называемых «ландшафтов террора» — т.е. концентрационных лагерей или массовых захоронений. Они позволяют восстановить детали, которые не отмечают письменные источники, как официальные документы, так и личные воспоминания. Третий пример не относится к научной археологии, но показывает наличие интереса общества к раскопкам. В частности, появления такого явления как геокэшинг, предполагающего поиск обычными людьми, «неархеологами», специально спрятанных «сокровищ» в определенных местах, имеющих историческую значимость. Этот пример свидетельствует о том, что общество нуждается в возможности участвовать в раскопках и получать о них информацию.

Приведенные примеры показывают, что тенденции, как в самой археологии, так и в обществе связаны с возобновлением интереса к эмпирическому познанию, центральная роль в развитии которого, безусловно, должна принад-

лежать раскопкам.

В связи с ростом количества проводимых археологических экспедиций возникает необходимость создания информационной системы, позволяющей упорядочить и минимизировать время сбора необходимых данных для ее организации.

Цель работы — разработка базы данных приложения по организации археологических экспедиций.

Для достижения поставленной цели необходимо решить следующие задачи:

- формализовать задачу и данные;
- спроектировать базу данных;
- выбрать систему управления базами данных;
- реализовать спроектированную базу данных и необходимый интерфейс для взаимодействия с ней;
- исследовать зависимость времени выполнения запроса от наличия индексов.

1 Аналитический раздел

1.1 Формализация задачи

Требуется разработать базу данных приложения по организации археологических экспедиций. В базе будет храниться информация об экспедициях, руководителях, кураторах и участниках экспедиций, местах раскопок, найденных там артефактах и необходимом для экспедиций снаряжении. Также необходимо разработать интерфейс для взаимодействия с базой данных.

Необходимо предусмотреть возможность создавать экспедиции с указанием места проведения, временных рамок, кураторов, руководителей и участников. Требуется реализовать три вида ролей — участник экспедиции, руководитель и администратор. Реализовать возможность для пользователя просматривать, создавать экспедиции и добавлять других участников в зависимости от его роли.

1.2 Формализация данных

База данных должна содержать информацию о следующих сущностях:

- экспедициях;
- руководителях экспедиций;
- участниках экспедиций;
- местах проведения экспедиций;
- кураторах экспедиций;
- снаряжении для экспедиций;
- найденных артефактах.

В таблице 1 приведены данные сущности и сведения о них.

Таблица 1 – Сущности и сведения о них

Сущность	Сведения
Локация	ID локации, название, страна, ближайший населенный пункт
Экспедиция	ID экспедиции, ID локации, дата начала экспедиции, дата окончания экспедиции
Участник	ID участника, имя, номер телефона, роль в экспедиции, логин, пароль
Руководитель	ID руководителя, имя, номер телефона, логин, пароль
Куратор	ID куратора, название
Снаряжение	ID снаряжения, ID экспедиции, название, количество
Артефакт	ID артефакта, ID локации, название, возраст

На рисунке 1 представлена ER-диаграмма сущностей проектируемой базы данных в нотации Чена.

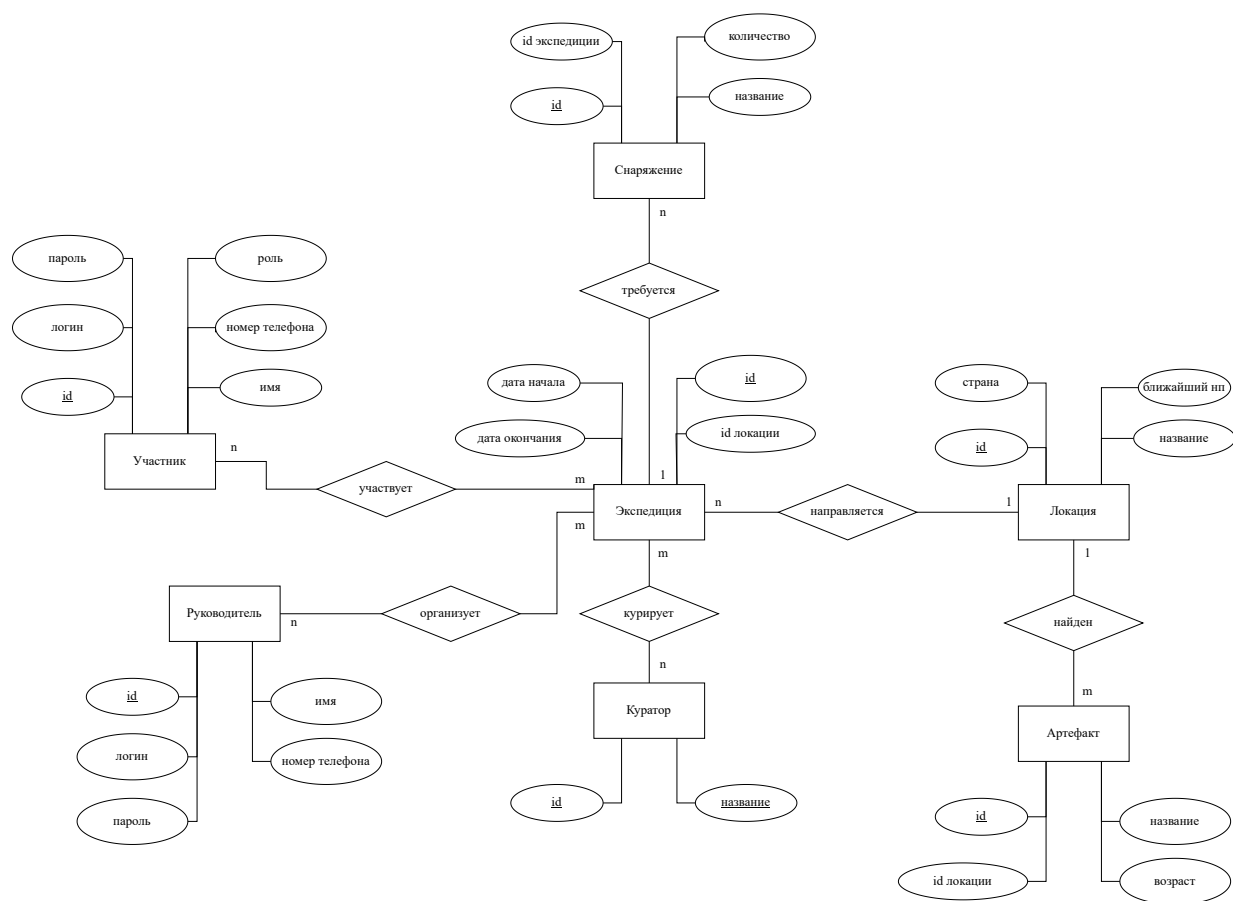


Рисунок 1 – ER-диаграмма

1.3 Формализация ролей

Для управления приложением выделено три типа пользователей: участник экспедиции, руководитель и администратор. Для каждой роли предусмотрен свой набор функций.

Набор функций участника:

- выйти из аккаунта;
- просмотр информации об экспедициях;
- просмотр информации о руководителях экспедиций;
- просмотр информации об участниках экспедиций;
- просмотр информации о кураторах экспедиций;
- просмотр информации о местах проведения экспедиций;
- просмотр информации о найденных артефактах;

— просмотр информации о снаряжении для экспедиций.

Набор функций руководителя:

- все возможности роли «участник»;
- добавление и удаление информации об участниках экспедиций;
- добавление, изменение и удаление информации об экспедициях;
- добавление и удаление информации о кураторах экспедиций;
- добавление и удаление информации о местах проведения экспедиций;
- добавление информации о найденных артефактах;
- добавление и удаление информации о снаряжении для экспедиций.

Набор функций администратора:

- все возможности роли «руководитель»;
- добавление и удаление информации о руководителях экспедиций.

Диаграмма вариантов использования приложения для выделенных ролей представлена на рисунке 2.

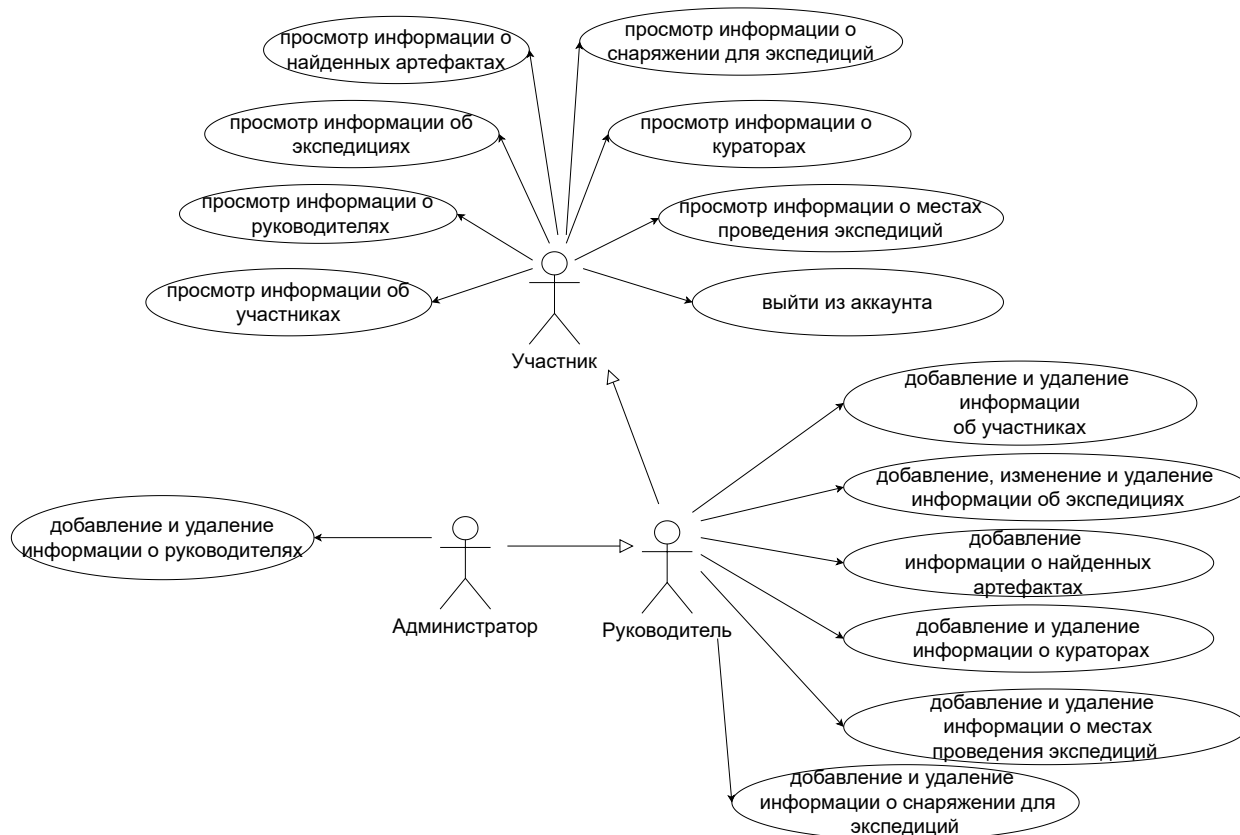


Рисунок 2 – Диаграмма вариантов использования

1.4 Базы данных и системы управления базами данных

База данных — это упорядоченный набор структурированной информации или данных, которые обычно хранятся в электронном виде в компьютерной системе [1].

Для базы данных обычно требуется комплексное программное обеспечение, которое называется системой управления базами данных (СУБД). СУБД основываются на различных моделях данных, которые определяют, каким образом данные будут храниться и обрабатываться внутри СУБД. Существуют три основных типа моделей данных, отличающиеся структурой организации данных — дореляционные, реляционные и постреляционные.

1.4.1 Дореляционные

Ранние модели называются дореляционными [2]. К ним относятся следующие модели.

1. Инвертированные списки (файлы).
2. Иерархическая.
3. Сетевая.

База данных на основе инвертированных списков представляет собой совокупность файлов, содержащих записи (таблиц). Для записей в файле определен порядок, диктуемый физической организацией данных. На основе инвертированных списков для каждого файла может быть определено произвольное число других упорядочений.

Иерархическая модель состоит из объектов с указателями от родительских объектов к потомкам, соединяя вместе связанную информацию. Такие базы данных могут быть представлены как дерево.

К основным понятиям сетевой модели относятся: элемент (узел) и связь. Узел представляет собой совокупность атрибутов данных, описывающих некоторый объект. Сетевая база данных может быть представлена в виде графа. Сетевая модель не является полностью независимой от приложений, так как ло-

гика выборки данных зависит от физической организации этих данных [3].

1.4.2 Реляционные

Реляционные базы данных основаны на реляционной модели — табличном способе представления данных. Каждая строка, содержащаяся в таблице, представляет собой запись с уникальным идентификатором, который называют ключом. Столбцы таблицы имеют атрибуты данных, а каждая запись обычно содержит значение для каждого атрибута, что дает возможность легко устанавливать взаимосвязь между элементами данных [4].

Для связывания информации из разных таблиц используются внешние ключи — уникальные идентификаторы атомарного фрагмента данных в этой таблице. Другие таблицы могут ссылаться на этот внешний ключ, чтобы создать связь между частями данных и частью, на которую указывает внешний ключ.

Реляционные базы данных также способны удовлетворять свойствам ACID: атомарность, согласованность, изолированность, надежность.

1.4.3 Постреляционные

Постреляционная модель является расширенной версией реляционной модели и позволяет устранить ограничение неделимости данных, хранящихся в записях таблиц. Постреляционная модель данных допускает многозначные поля — поля, значения которых состоят из подзначений. Набор значений многозначных полей считается самостоятельной таблицей, встроенной в основную таблицу [5].

1.4.4 Выбор модели данных

В соответствии с формализацией данных можно выделить следующие требования к разрабатываемой базе данных:

- поддержка принципов ACID;
- чётко установленные схемы для данных.

Исходя из требований к разрабатываемой базе данных можно сделать вывод о том, что для решения задачи предпочтительной является реляционная

модель данных.

Вывод

В данной части были определены сущности разрабатываемой базы данных, выделены роли для управления приложением и описаны функции каждой из ролей. Был произведен анализ моделей данных, для реализации была выбрана реляционная модель.

2 Конструкторский раздел

2.1 Проектирование базы данных

На рисунке 3 представлена диаграмма проектируемой базы данных.

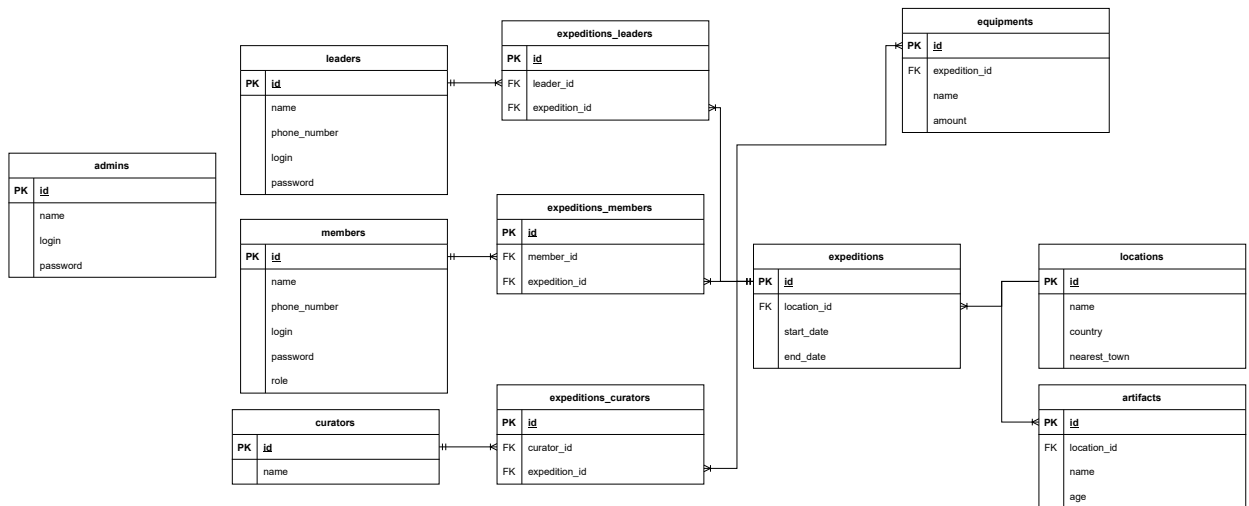


Рисунок 3 – Диаграмма базы данных

Таблица **leaders** хранит информацию о руководителях экспедиций и содержит следующие поля:

1. **id** — идентификатор руководителя, **int**;
2. **name** — имя руководителя, **text**;
3. **phone_number** — номер телефона, **text**;
4. **login** — логин руководителя, **text**;
5. **password** — пароль, **text**.

Таблица **members** хранит информацию об участниках экспедиций и содержит следующие поля:

1. **id** — идентификатор участника, **int**;
2. **name** — имя участника, **text**;
3. **phone_number** — номер телефона, **text**;
4. **role** — роль участника в экспедиции, **text**;
5. **login** — логин участника, **text**;
6. **password** — пароль, **text**.

Таблица `admins` хранит информацию об администраторах и содержит следующие поля:

1. `id` — идентификатор администратора, `int`;
2. `name` — имя администратора, `text`;
3. `login` — логин администратора, `text`;
4. `password` — пароль, `text`.

Таблица `curators` хранит информацию о кураторах экспедиций и содержит следующие поля:

1. `id` — идентификатор куратора, `int`;
2. `name` — название, `text`.

Таблица `locations` хранит информацию о местах проведения экспедиций и содержит следующие поля:

1. `id` — идентификатор локации, `int`;
2. `name` — название локации, `text`;
3. `country` — страна, `text`;
4. `nearest_town` — ближайший населенный пункт, `text`.

Таблица `expeditions` хранит информацию об экспедициях и содержит следующие поля:

1. `id` — идентификатор экспедиции, `int`;
2. `location_id` — идентификатор места проведения экспедиции, `int`;
3. `start_date` — дата начала экспедиции, `date`;
4. `end_date` — дата окончания экспедиции, `date`.

Таблица `artifacts` хранит информацию о найденных артефактах и содержит следующие поля:

1. `id` — идентификатор артефакта, `int`;
2. `location_id` — идентификатор места обнаружения артефакта, `int`;
3. `name` — название артефакта, `text`;
4. `age` — возраст артефакта, `int`.

Таблица `equipments` хранит информацию о снаряжении для экспедиции и

содержит следующие поля:

1. id — идентификатор снаряжения, int;
2. expedition_id — идентификатор экспедиции, int;
3. name — название снаряжение, text;
4. amount — количество снаряжения, int.

Таблица `expeditions_leaders` реализует связь «многие-ко-многим» таблиц `expeditions` и `leaders` и содержит следующие поля:

1. id — идентификатор записи, int;
2. expedition_id — идентификатор экспедиции, int;
3. leader_id — идентификатор руководителя, int.

Таблица `expeditions_members` реализует связь «многие-ко-многим» таблиц `expeditions` и `members` и содержит следующие поля:

1. id — идентификатор записи, int;
2. expedition_id — идентификатор экспедиции, int;
3. member_id — идентификатор участника, int.

Таблица `expeditions_curators` реализует связь «многие-ко-многим» таблиц `expeditions` и `curators` и содержит следующие поля:

1. id — идентификатор записи, int;
2. expedition_id — идентификатор экспедиции, int;
3. curator_id — идентификатор куратора, int.

2.2 Триггер базы данных

Для добавления нового участника к экспедиции необходимо убедиться, что он не участвует в других экспедициях, пересекающихся по времени проведения с текущей. Для этого необходим триггер. При попытке добавления участника к экспедиции в развязочной таблице для участников и экспедиций вызывается функция, схема алгоритма работы которой представлена на рисунке 4.

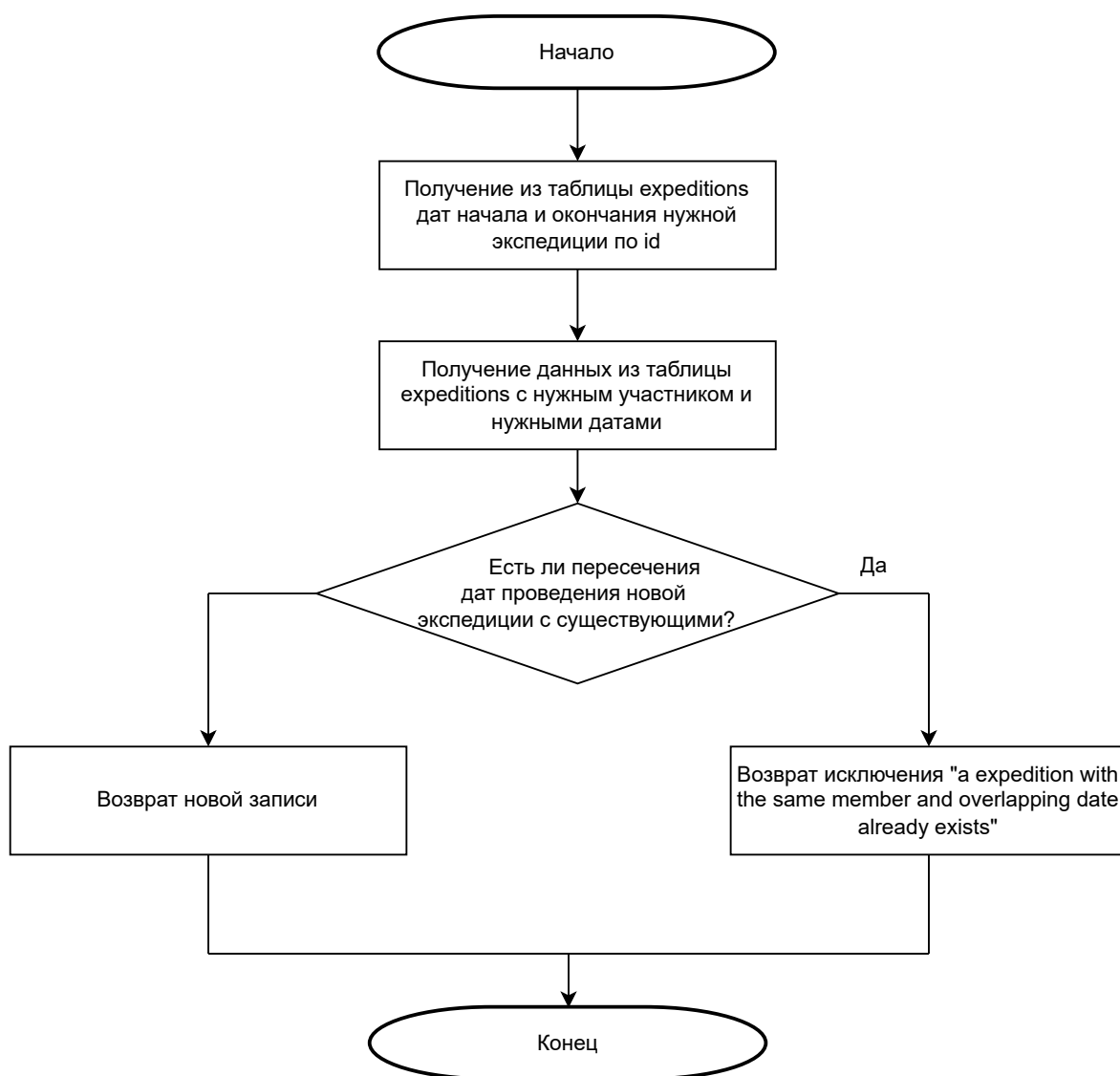


Рисунок 4 – Схема алгоритма работы функции `check_expedition_dates()`

2.3 Роли базы данных

Разрабатываемая база данных имеет следующие роли.

1. Участник — права на выборку из таблиц `expeditions`, `leaders`, `members`, `curators`, `locations`, `artifacts`, `equipments`, `expeditions_leaders`, `expeditions_members`, `expeditions_curators`.
2. Руководитель — права на выборку из таблиц `expeditions`, `leaders`, `members`, `curators`, `locations`, `artifacts`, `equipments`, `expeditions_leaders`, `expeditions_members`, `expeditions_curators`; вставку в таблицы `members`,

expeditions, curators, locations, artifacts, equipments, expeditions_members, expeditions_curators; удаление из таблиц members, expeditions, curators, locations equipments, expeditions_members, expeditions_curators; изменение в таблице expeditions.

3. Администратор — права на все операции над всеми таблицами, создание таблиц и ролей.

Вывод

В данной части были описаны сущности, ролевая модель и функция базы данных.

3 Технологический раздел

3.1 Выбор СУБД

При выборе СУБД для решения поставленной задачи можно выделить следующие критерии:

- открытый исходный код;
- опыт работы;
- возможность создания триггеров.

Результаты сравнения наиболее популярных СУБД по указанным критериям приведены в таблице 2.

Таблица 2 – Сравнение СУБД

СУБД	Критерий 1	Критерий 2	Критерий 3
PostgreSQL [6]	+	+	+
MySQL [7]	+	-	+
Oracle [8]	-	-	+

По результатам сравнения в данной работе будет использоваться СУБД PostgreSQL.

3.2 Средства реализации

Для реализации приложения использовались следующие технологии.

1. Язык программирования — Golang [9].
2. Система управления базами данных — PostgreSQL.
3. Драйвер для работы с СУБД PostgreSQL — pgx [10].
4. Фреймворк для реализации API — gin [11].
5. ПО для развертывания приложения — Docker [12].
6. Фреймворк для документации REST API — Swagger [13].

3.3 Детали реализации

3.3.1 Таблицы базы данных

В листингах 1—4 представлен код создания таблиц, спроектированных в конструкторской части.

Листинг 1: Скрипт создания таблиц (часть 1)

```
1 create table if not exists admins
2 (
3     id            int generated always as identity primary key,
4     name          text not null ,
5     login         text unique not null ,
6     password      text not null
7 );
8
9 create table if not exists leaders
10 (
11     id            int generated always as identity primary key,
12     name          text not null ,
13     phone_number  text not null ,
14     login         text unique not null ,
15     password      text not null
16 );
17
18 create table if not exists members
19 (
20     id            int generated always as identity primary key,
21     name          text not null ,
22     phone_number  text not null ,
23     role          text not null ,
24     login         text unique not null ,
25     password      text not null
26 );
```

Листинг 2: Скрипт создания таблиц (часть 2)

```
1 create table if not exists curators
2 (
3     id            int generated always as identity primary key,
4     name          text unique not null
5 );
6
7 create table if not exists locations
8 (
9     id            int generated always as identity primary key,
10    name          text not null ,
11    country       text not null ,
12    nearest_town  text not null
13 );
14
15 create table if not exists expeditions
16 (
17     id            int generated always as identity primary key,
18     location_id   int not null ,
19     start_date    date not null ,
20     end_date      date not null ,
21
22     foreign key (location_id) references locations(id)
23     on delete cascade
24 );
```

Листинг 3: Скрипт создания таблиц (часть 3)

```
1 create table if not exists artifacts
2 (
3     id          int generated always as identity primary key,
4     location_id int not null ,
5     name        text not null ,
6     age         int not null ,
7
8     foreign key (location_id) references locations(id)
9     on delete cascade
10 );
11
12 create table if not exists equipments
13 (
14     id          int generated always as identity primary key,
15     expedition_id int not null ,
16     name        text not null ,
17     amount      int not null ,
18
19     foreign key (expedition_id) references expeditions(id)
20     on delete cascade
21 );
22
23 create table if not exists expeditions_leaders
24 (
25     id          int generated always as identity primary key,
26     expedition_id int not null ,
27     leader_id   int not null ,
28
29     foreign key (expedition_id) references expeditions(id)
30     on delete cascade,
31     foreign key (leader_id) references leaders(id)
32     on delete cascade
33 );
```

Листинг 4: Скрипт создания таблиц (часть 4)

```
1 create table if not exists expeditions_members
2 (
3     id            int generated always as identity primary key,
4     expedition_id int not null ,
5     member_id     int not null ,
6
7     foreign key (expedition_id) references expeditions(id)
8     on delete cascade,
9     foreign key (member_id) references members(id)
10    on delete cascade
11 );
12
13 create table if not exists expeditions_curators
14 (
15     id            int generated always as identity primary key,
16     expedition_id int not null ,
17     curator_id    int not null ,
18
19     foreign key (expedition_id) references expeditions(id)
20     on delete cascade,
21     foreign key (curator_id) references curators(id)
22     on delete cascade
23 );
```

3.3.2 Роли базы данных

В листинге 5 представлен код создания ролей и выделения им прав, соответствующих разработанной в конструкторской части ролевой модели.

Листинг 5: Скрипт создания ролевой модели базы данных

```
1 create role member;
2 grant select on public.expeditions to member;
3 grant select on public.leaders to member;
4 grant select on public.members to member;
5 grant select on public.curators to member;
6 grant select on public.locations to member;
7 grant select on public.artifacts to member;
8 grant select on public.equipments to member;
9 grant select on public.expeditions_leaders to member;
10 grant select on public.expeditions_members to member;
11 grant select on public.expeditions_curators to member;
12
13 create user member1 with PASSWORD 'member1' in role member;
14
15 create role leader inherit;
16 grant member to leader;
17 grant insert, delete on public.members to leader;
18 grant insert, update, delete on public.expeditions to leader;
19 grant insert, delete on public.curators to leader;
20 grant insert, delete on public.locations to leader;
21 grant insert on public.artifacts to leader;
22 grant insert, delete on public.equipments to leader;
23 grant insert, delete on public.expeditions_members to leader;
24 grant insert, delete on public.expeditions_curators to leader;
25
26 create user leader1 with PASSWORD 'leader1' in role leader;
27
28 create role admin;
29 grant create, usage on schema public to admin;
30 grant all privileges on all tables in schema public to admin;
31
32 create user admin1 with PASSWORD 'admin1' in role admin;
```

3.3.3 Триггер базы данных

В листингах 6, 7 представлен код триггера и соответствующей функции, разработанной в конструкторской части.

Листинг 6: Триггер базы данных (часть 1)

```
1 create or replace function check_expedition_dates()
2 returns trigger as $$
3 declare
4     start_d date;
5     end_d date;
6     overlapping_count integer;
7 begin
8     select start_date , end_date
9     into start_d , end_d
10    from expeditions
11    where id = new.expedition_id;
12
13     select count(*)
14     into overlapping_count
15    from expeditions ex
16    join expeditions_members em on ex.id = em.expedition_id
17    where em.member_id = new.member_id and
18           not(end_d < ex.start_date or start_d > ex.end_date);
19
20     if overlapping_count > 0 then
21         raise exception
22             'a expedition with the same member ' ||
23             'and overlapping date already exists ';
24     end if;
25
26     return new;
27 end;
28 $$ language plpgsql;
```

Листинг 7: Триггер базы данных (часть 2)

```
1 create or replace trigger check_expedition_dates_trigger
2 before insert on expeditions_members
3 for each row
4 execute function check_expedition_dates();
```

3.4 Тестирование

Тестирование компонента бизнес—логики выполнялось с использованием пакета Testing [14]. Для тестирования были написаны модульные тесты с использованием библиотеки Gomock [15], которая позволяет использовать заглушки, с помощью которых были реализованы тесты.

Так же были написаны интеграционные тесты для связи компонента бизнес—логики и компонента доступа к данным с использованием пакета Testing.

3.5 Интерфейс приложения

Приложение является web—сервером, взаимодействие с которым может быть реализовано через интерфейс REST API. Описание REST API приложения представлено в таблице 3.

Таблица 3 – Описание REST API реализуемого приложения

Путь	HTTP-метод	Описание
/api/v1/login	POST	Вход в систему
/api/v1/logout	POST	Выход из системы
/api/v1/leaders	GET	Получение всех руководителей
/api/v1/leader/{expedition_id}	GET	Получение руководителей заданной экспедиции
/api/v1/leader	POST	Добавление руководителя
/api/v1/leader_expedition	POST	Добавление связи руководителя и экспедиции
/api/v1/leader/{id}	DELETE	Удаление руководителя по идентификатору
/api/v1/members	GET	Получение всех участников
/api/v1/member/{expedition_id}	GET	Получение участников заданной экспедиции
/api/v1/member	POST	Добавление участника
/api/v1/member_expedition	POST	Добавление связи участника и экспедиции
/api/v1/member/{id}	DELETE	Удаление участника по идентификатору
/api/v1/curators	GET	Получение всех кураторов экспедиций
/api/v1/curator/{expedition_id}	GET	Получение кураторов заданной экспедиции
Продолжение на следующей странице		

Таблица 3 – продолжение

Путь	Метод	Описание
/api/v1/curator	POST	Добавление куратора
/api/v1/curator_expedition	POST	Добавление связи куратора и экспедиции
/api/v1/curators/{id}	DELETE	Удаление куратора по идентификатору
/api/v1/admins	GET	Получение всех администраторов
/api/v1/admin	POST	Добавление администратора
/api/v1/admin/{id}	DELETE	Удаление администратора по идентификатору
/api/v1/locations	GET	Получение всех мест проведения экспедиций
/api/v1/location	POST	Добавление места проведения экспедиции
/api/v1/location/{id}	DELETE	Удаление места проведения экспедиции по идентификатору
/api/v1/expeditions	GET	Получение всех экспедиций
/api/v1/expedition/{location_id}	GET	Получение экспедиций, проведенных в заданной локации
/api/v1/expedition	POST	Добавление экспедиции
/api/v1/expedition/{id}	PATCH	Редактирование дат проведения экспедиции по идентификатору
/api/v1/expedition/{id}	DELETE	Удаление экспедиции по идентификатору
/api/v1/artifacts	GET	Получение всех найденных артефактов
Продолжение на следующей странице		

Таблица 3 – продолжение

Путь	Метод	Описание
/api/v1/artifact/{location_id}	GET	Получение артефактов, найденных в заданной локации
/api/v1/artifact	POST	Добавление артефакта
/api/v1/equipments	GET	Получение всего снаряжения
/api/v1/equipment/{expedition_id}	GET	Получение снаряжения заданной экспедиции
/api/v1/equipment	POST	Добавление снаряжения
/api/v1/equipment/{id}	DELETE	Удаление снаряжения по идентификатору
Конец таблицы		

3.6 Примеры работы ПО

Для документации REST API был использован фреймворк Swagger [13], который так же позволяет отправлять запросы к приложению. На рисунках 5 и 6 представлены примеры.

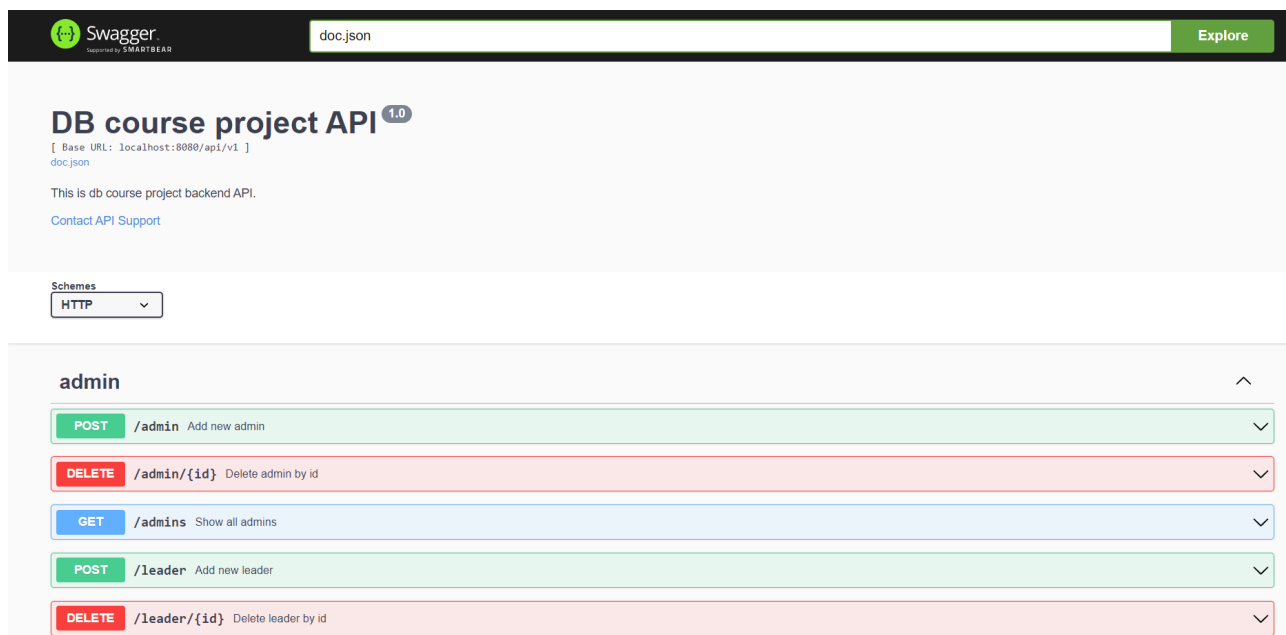


Рисунок 5 – Главная страница

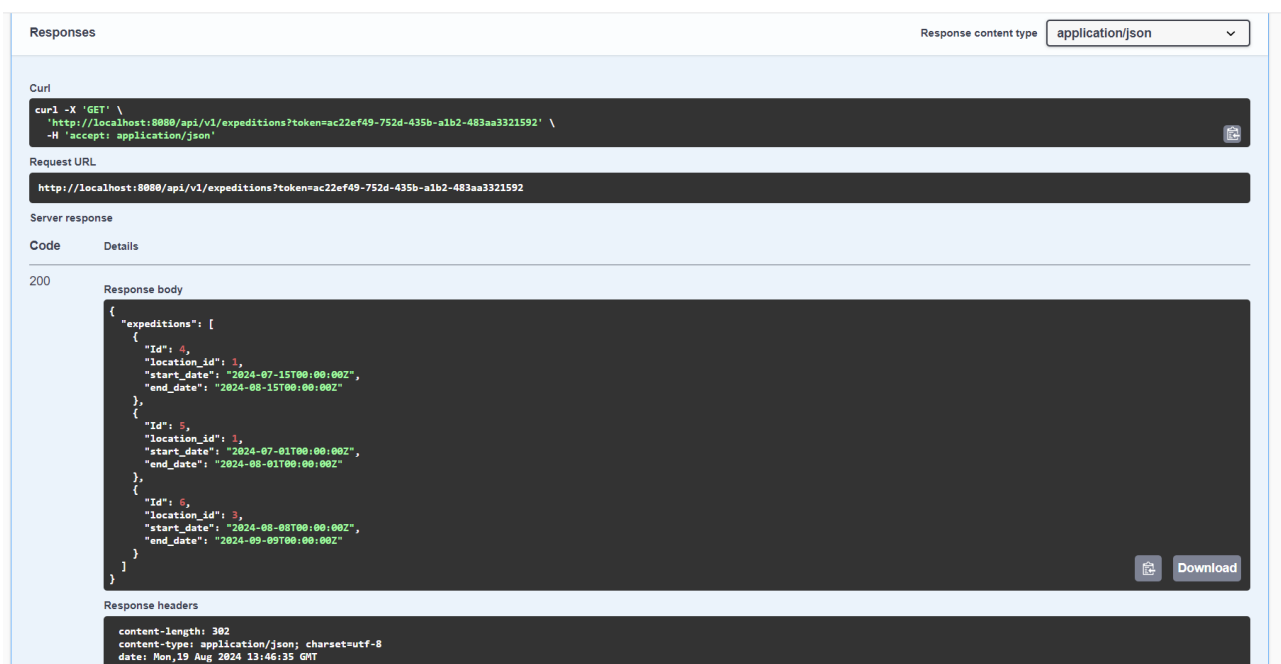


Рисунок 6 – Пример ответа на запрос получения всех экспедиций

Вывод

В данном разделе был проведен анализ и выбор СУБД, описаны средства и детали реализации приложения, описан интерфейс взаимодействия с приложением.

4 Исследовательский раздел

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование, следующие:

- операционная система: Windows 11;
- оперативная память: 16 Гб;
- процессор: Intel Core™ i5—1135G7.

Тестирование проводилось на ноутбуке, который был включен в сеть питания. Во время тестирования ноутбук был нагружен только встроенными приложениями окружения и системой тестирования.

4.2 Цель и описание исследования

Целью исследования является исследование зависимости времени выполнения запроса от наличия индексов.

Исследование проводилось на таблице, содержащих информацию об экспедициях, так как данная таблица является реализацией основной сущности в системе. Для исследования был выбран запрос на поиск всех экспедиций, проведенных в указанной локации. Тестировалось время выполнения запроса при количестве записей в таблице экспедиций от 100 до 10000. Запрос поиска приведен на листинге 8.

Листинг 8: Запрос поиска

```
1 EXPLAIN ANALYZE SELECT id , location_id , start_date , end_date
2 FROM expeditions
3 WHERE location_id = 50
```

Создание индекса приведено на листинге 9.

Листинг 9: Создание индекса

```
1 CREATE INDEX idx_expeditions_location_id
2 ON expeditions(location_id);
```

Примеры выполнения запроса с индексацией и без представлены на рисунках 7 и 8 соответственно.

ABC QUERY PLAN
Bitmap Heap Scan on expeditions (cost=5.11..61.91 rows=106 width=16) (actual time=0.028..0.088 rows=106 loops=1)
Recheck Cond: (location_id = 50)
Heap Blocks: exact=45
-> Bitmap Index Scan on idx_expeditions_location_id (cost=0.00..5.08 rows=106 width=0) (actual time=0.019..0.019 rows=106 loops=1)
Index Cond: (location_id = 50)
Planning Time: 0.179 ms
Execution Time: 0.103 ms

Рисунок 7 – Поиск с использованием индексирования

ABC QUERY PLAN
Seq Scan on expeditions (cost=0.00..180.00 rows=106 width=16) (actual time=0.026..0.657 rows=106 loops=1)
Filter: (location_id = 50)
Rows Removed by Filter: 9894
Planning Time: 0.147 ms
Execution Time: 0.677 ms

Рисунок 8 – Поиск без использования индексирования

4.3 Результаты исследования

График зависимости времени поиска от размера таблицы экспедиций и использования индексирования, представлен на рисунке 9.

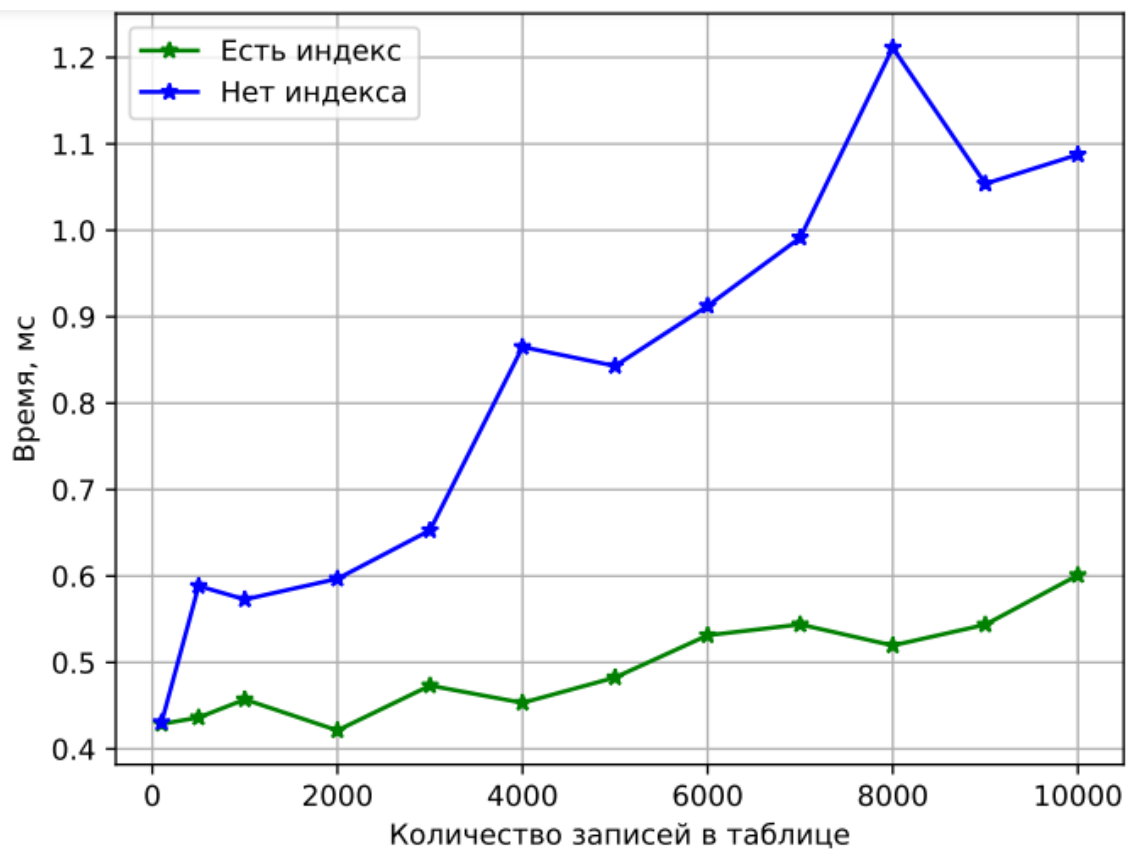


Рисунок 9 – Зависимость времени выполнения поиска от размера таблицы и использования индексирования

Вывод

В данном разделе было проведено исследование зависимости времени выполнения запроса от наличия индексов. В результате исследования было выявлено, что использование индексов в среднем уменьшает время выполнения запроса в 1,5—2 раза.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы достигнута поставленная цель: разработана база данных приложения по организации археологических экспедиций.

Были решены все поставленные задачи:

- формализована задача и данные;
- спроектирована база данных;
- выбрана система управления базами данных;
- реализована спроектированная база данных и интерфейс для взаимодействия с ней;
- проведено исследование зависимости времени выполнения запроса от наличия индексов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Что такое база данных | Oracle Россия и СНГ [Электронный ресурс]. — Режим доступа: <https://www.oracle.com/ru/database/what-is-database/> (дата обращения: 18.08.2024).
2. Сергеева Т.И. «Базы данных: модели данных, проектирование, язык SQL: учебное пособие» — Воронеж: ФГБОУ ВПО «Воронежский государственный технический университет», 2012. — 233 с.
3. Томас Коннолли, Каролин Бегг. «Базы данных: Проектирование, реализация и сопровождение. Теория и практика», 3-е издание, издательский дом «Вильямс», 2017. — 1440 с.
4. Что такое реляционная база данных | Oracle СНГ [Электронный ресурс]. — Режим доступа: <https://www.oracle.com/cis/database/what-is-a-relational-database/> (дата обращения: 18.08.2024).
5. Постреляционная модель базы данных [Электронный ресурс]. — Режим доступа: http://db4.sbras.ru/elbib/data/show_page.phtml?20+1588 (дата обращения: 18.08.2024).
6. PostgreSQL: The World's Most Advanced Open Source Relational Database [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/> (дата обращения: 03.09.2024).
7. MySQL – the world's most popular open source database [Электронный ресурс]. — Режим доступа: <https://www.mysql.com/> (дата обращения: 03.09.2024).
8. Oracle database services and products offer customers cost-optimized and high-performance versions of Oracle Database, the world's leading converged, multi-model database management system. [Электронный ресурс]. — Режим доступа: <https://www.oracle.com/database/> (дата обращения: 03.09.2024).

9. Golang — компилируемый многопоточный язык программирования [Электронный ресурс]. — Режим доступа: <https://pkg.go.dev/std/> (дата обращения: 03.09.2024).
10. pgx — PostgreSQL Driver and Toolkit [Электронный ресурс]. — Режим доступа: <https://pkg.go.dev/github.com/jackc/pgx/v5> (дата обращения: 03.09.2024).
11. Gin Web Framework. The fastest full-featured web framework for Go. [Электронный ресурс]. — Режим доступа: <https://gin-gonic.com/> (дата обращения: 04.09.2024).
12. Docker is a platform designed to help developers build, share, and run modern application. [Электронный ресурс]. — Режим доступа: <https://docs.docker.com/> (дата обращения: 05.09.2024).
13. Swagger: API Documentation & Design Tools for Teams [Электронный ресурс]. — Режим доступа: <https://swagger.io/> (дата обращения: 03.09.2024).
14. Testing package documentation [Электронный ресурс]. — Режим доступа: <https://pkg.go.dev/testing> (дата обращения: 03.09.2024).
15. Gomock package documentation [Электронный ресурс]. — Режим доступа: <https://pkg.go.dev/github.com/golang/mock/gomock> (дата обращения: 03.09.2024).

ПРИЛОЖЕНИЕ А

Презентация